Docker Multi-Stage Build for a React Application

Overview

Docker Multi-Stage Builds provide an efficient way to create production-ready container images. Instead of including all development dependencies in the final image, the application is built in one stage and only the required build artifacts are copied into a lightweight production image.

In this tutorial, you will build a React application using Node.js and deploy it with Nginx using a Docker Multi-Stage Build. The final image will contain only the production-ready files, resulting in a smaller and more secure container.

Technology Stack

Ubuntu Server	Development environment
Docker	Containerization platform
React.js	Frontend application
Node.js	Build environment
Nginx	Production web server

Objectives

- Create a React application for deployment
- Deploy the application's static files using Nginx
- Optimize the Docker image using a Multi-Stage Build
- Create a lightweight production image by excluding unnecessary build dependencies

Prerequisites

Ensure that Docker, Node.js, and npm are installed on the Ubuntu server before proceeding.

- Install Docker

```
# sudo apt install docker.io -y
```

- Verify docker installation

```
root@ip-172-31-33-66:/home/ubuntu# docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
```

- Install Node.js

```
# curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
```

```
# sudo apt install -y nodejs
```

- Verify node.js installation

```
root@ip-172-31-33-66:/home/ubuntu# node --version
v22.23.1
root@ip-172-31-33-66:/home/ubuntu# npm --version
10.9.8
```

Step 1: Create React Application

- Create a working directory

```
# mkdir react-docker-demo
```

```
# cd react-docker-demo
```

- Create a React Application using vite

```
# npm create vite@latest react-app
```

- Select the following options when prompted

  Framework: React

  Variant: JavaScript

- Navigate to the project directory and install the dependencies.

```
# cd react-app
```

```
# npm install
```

- After installation, the project structure should resemble the following

```
root@ip-172-31-33-66:/home/ubuntu/react-docker-demo/react-app# ls -l
total 116
-rw-r--r--   1 root root  1027 Jun 28 08:44 README.md
-rw-r--r--   1 root root   568 Jun 28 08:44 eslint.config.js
-rw-r--r--   1 root root   361 Jun 28 08:44 index.html
drwxr-xr-x 110 root root  4096 Jun 28 08:47 node_modules
-rw-r--r--   1 root root 83442 Jun 28 08:47 package-lock.json
-rw-r--r--   1 root root   605 Jun 28 08:44 package.json
drwxr-xr-x   2 root root  4096 Jun 28 08:44 public
drwxr-xr-x   3 root root  4096 Jun 28 09:04 src
-rw-r--r--   1 root root   161 Jun 28 08:44 vite.config.js
```

Step 2: Configure the Project for Docker

- Create a **.dockerignore** file to exclude unnecessary files from the build context. This reduces the image size and improves build performance.

```
# touch .dockerignore
```

- Open the file and add the following entries

```
# nano .dockerignore
```

```
node_modules
.git
.gitignore
README.md
dist
```

- The project structure should now look similar to the following.

```
root@ip-172-31-33-66:/home/ubuntu/react-docker-demo/react-app# ls -la
total 132
drwxr-xr-x   5 root root  4096 Jun 28 09:20 .
drwxr-xr-x   3 root root  4096 Jun 28 09:20 ..
-rw-r--r--   1 root root    44 Jun 28 09:20 .dockerignore
-rw-r--r--   1 root root   253 Jun 28 08:44 .gitignore
-rw-r--r--   1 root root  1027 Jun 28 08:44 README.md
-rw-r--r--   1 root root   568 Jun 28 08:44 eslint.config.js
-rw-r--r--   1 root root   361 Jun 28 08:44 index.html
drwxr-xr-x 110 root root  4096 Jun 28 08:47 node_modules
-rw-r--r--   1 root root 83442 Jun 28 08:47 package-lock.json
-rw-r--r--   1 root root   605 Jun 28 08:44 package.json
drwxr-xr-x   2 root root  4096 Jun 28 08:44 public
drwxr-xr-x   3 root root  4096 Jun 28 09:04 src
-rw-r--r--   1 root root   161 Jun 28 08:44 vite.config.js
```

Step 3: Create the Multi-Stage Dockerfile

- Create the Dockerfile.

```
# touch Dockerfile
```

- Open the Dockerfile.

```
# nano Dockerfile
```

- Paste the following code

```
# Stage 1 - Build Stage
FROM node:22 AS builder

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build

# Stage 2 - Production Stage
FROM nginx:alpine

COPY --from=builder /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Step 4: Build and Run the Docker Image

- Build the Docker image

```
# docker build -t react-docker-app .
```

- Verify that the image has been created.

```
# docker images
```

```
IMAGE                          ID             DISK USAGE   CONTENT SIZE   EXTRA
nginx:alpine                   54f2a904c251       93.3MB         26.9MB
node:22                        c601a46abb4d       1.64GB          425MB
react-docker-app:latest        1e6deca558c8       92.8MB         26.1MB    U
```

- Run the container

```
# docker run -d -p 8080:80 --name react-app react-docker-app
```

- Verify the container is running

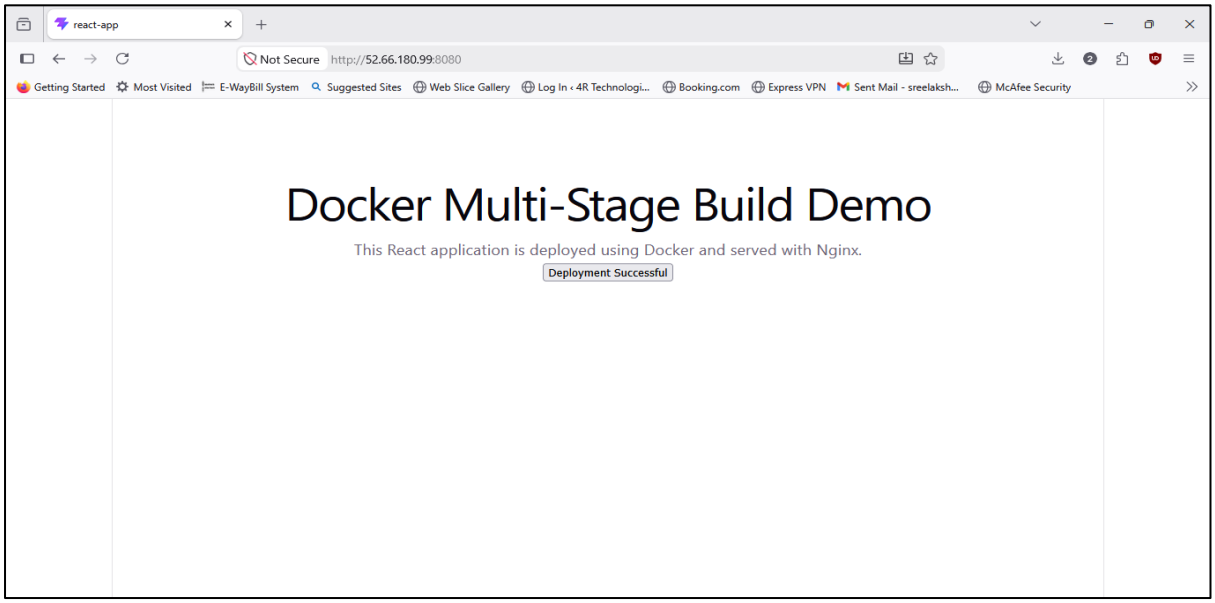
```
# docker ps
```

```
CONTAINER ID   IMAGE              COMMAND                  CREATED         STATUS         PORTS                                       NAMES
71a2abfc887e   react-docker-app   "/docker-entrypoint.…"   8 seconds ago   Up 7 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   react-app
```

Step 5: Verify the Deployment

Open a web browser and navigate to:

http://<EC2-Public-IP>:8080



Step 6: Cleanup

- Stop the running container.

```
# docker stop react-app
```

- Remove the container.
```
# docker rm react-app
```

- Remove the Docker image
```
# docker rmi react-docker-app
```

Conclusion

In this tutorial, a simple React application was created and containerized using a Docker Multi-Stage Build. The application was built using Node.js in the first stage and deployed using Nginx in the second stage. By excluding development dependencies from the final image, the resulting container is lightweight, efficient, and suitable for production deployments.